able to work in an environment with near-instantaneous test feedback, you know it's an entirely different level of dealing with tests.

Worse, there's a slow acclimation that comes with a gradually increasing test suite. You'll add only a little bit of time to the test suite every day—practically nothing. But weeks and months of practically nothing being added lead you not to notice that suddenly you have time to go get coffee during your test run. Preventing this gradual time creep takes vigilance.

I'm not saying that your entire test suite needs to run in 15 seconds. That's not feasible for most applications, and it's probably not a good idea in any case—you need some of those slow integration tests. I'm saying that the value of your testing will improve if you have a reliable way to run relevant tests in a few seconds. That way might be Guard, it might include Spring, or RSpec metadata, or the `--only-failures` options, but it's worth making sure you have a way of doing so. It's also true that it's much easier to isolate a subset of tests if your application is broken into relatively distinct sections.

Prescription 48 If you have enough time to break focus while your tests run, you aren't getting the full value of the TDD process.

Why Separation from Rails Is Useful

The primary advantage of writing your tests to avoid Rails is that doing so encourages you to structure your code to better manage increasing complexity and change over time. Super-fast tests are just a side benefit.

Here are some of the advantages, which are related to the idea that there is a minimum of coupling between any two parts of the code:

- It's easier to change one element of the code if the impact of that change is limited in how it affects other parts of the code. Limiting the access that code has to the internals of other sections makes it easier to change those internals. In Rails terms, ActiveRecord associations and attributes often qualify as internals since they're based on an implementation detail—the naming convention of the underlying database. Putting lots of logic in Rails controllers and models encourages intertwining potentially unrelated functionality.
- The framework imposes generic names on common actions, whereas it's usually valuable to name specific items in your application after domain concepts. For example, `order(year: :asc)` is less meaningful than `chronological`.